Available online at www.sciencedirect.com

ScienceDirect

IFAC PapersOnline CONFERENCE PAPER ARCHIVE

IFAC PapersOnLine 53-5 (2020) 1–6

Human-Robot Collaboration Multi-objective Disassembly Line Balancing Subject to Task Failure via Multi-objective Artificial Bee Colony Algorithm

Chengcheng Xu [1], Haiping Wei [1], XiWang Guo[1], *Member*, *IEEE*, ShiXin Liu[2], Liang Qi[3], *Member*, *IEEE*, ZiYan Zhao[2]

1. College of Computer and Communication Engineering, Liaoning Shihua University, Fushun, 113001 China (e-mail: 1297701989@qq.com, 416089317@qq.com, x.w.guo@163.com)

2. College of Information Science and Engineering, Northeastern University, Shenyang, 110819 China (e-mail: sxliu@mail.neu.edu.cn, neuzhaoziyan@163.com)

3. Department of Intelligent Sci. & Tech., Shandong University of Science and Technology, Qingdao, 266590 China (e-mail: qiliangsdkd@163.com)

Abstract: Disassembly is a crucial process of remanufacturing end-of-life products. It can be done by either humans or robots. Manual disassembly has a high cost and low disassembly efficiency, while a robot is not flexible to perform different dynamic, complicated tasks. The combination of them can improve the disassembly efficiency. Because it is hard to know the detailed information about the wear and tear of the used products, there is a risk of disassembly failure. In this work, disassembly failure risks, constraints of disassembly priority, cycle time, and actual cost are taken into full consideration based on an AND/OR graph of used products. Besides, a mathematical model is constructed with the objectives of maximizing the profit and minimizing consumption, disassembly difficulties, and the number of workstations. Then, a new multi-objective artificial bee colony algorithm is proposed, which is combined with stochastic simulation. The effectiveness and feasibility of the proposed algorithm are verified and experimental results show that the proposed algorithm outperforms both nondominated sorting genetic algorithm II and multi-objective discrete grey wolf optimization algorithm.

Keywords: Human-robot cooperation, Bee colony algorithm, Disassembly line balancing, Remanufacturing, Failure risk, Stochastic simulation, Multi-objective disassembly

I. INTRODUCTION

Product recycling has received much attention from both industry and academy. Fu *et al*. (2019) present that it plays an essential role in the treatment of end-of-life (EOL) products. The traditional disassembly processes are usually done manually. However, it is costly and inefficient. Wegener *et al*. (2015) describe that advanced technology is essential in this process; it can be done by robots. When a specific disassembly task is conducted, the working efficiency of the robots is much higher than that of the human in Xiong *et al*. (2020). However, when different complex disassembly tasks are executed, the robots cannot flexibly accomplish them, so a human-robot collaborative disassembly (HRCD) is proposed by Li *et al*. (2019), Tian *et al*. (2012) to improve disassembly efficiency and reduce disassembly cost. At present, most of the tasks are carried out by a single robot or human in Huang *et al*. (2019), Guo *et al*. (2019) and few studies concern the human-robot collaborative methods for solving disassembly line balancing problems (DLBPs).

DLBP has been widely studied in the past decades. Özceylan *et al*. (2018). McGovern and Gupta. (2007) propose a DLBP by minimizing the number of workstations and their idle time. Liu *et al*. (2018) propose an improved discrete bee algorithm to minimize the number of workstations, idle time of workstations, and demand-related criterion. Pistolesi *et al*. (2018) design a multi-objective genetic algorithm to minimize the number of workstations and maximize the total profit and the number of tasks in DLBP. Ilgin *et al*. (2017) focus on balancing disassembly tasks among workstations and design a linear physical programming-based disassembly line balancing method.

Different from the studies above considering deterministic DLBPs, uncertainty in a disassembly process has drawn much attention in many studies. Zhang *et al*. (2017) consider the delay in the actual disassembly environment and propose an improved Pareto artificial fish swarm algorithm (IAFSA). Zheng *et al*. (2018) focus on DLBP with stochastic time and design some strategies to minimize workstation task cost and hazardous component processing costs. Kalayci *et al*. (2015) deal with uncertain task-processing time in DLBP, apply triangular fuzzy membership functions to describe it, and design a hybrid discrete artificial bee colony algorithm to minimize the number of workstations, idle time of workstations, and hazard values. Compared with the prior studies, this paper has three contributions:

1) In order to improve the efficiency of disassembly and reduce the cost of disassembly, a Human-Robot Collaborative Disassembly Line Balance Problem (HC-DLBP) is proposed. It takes the relationship among components of a product, energy consumption, uncertainty, and disassembly failure into consideration. The objectives are to maximize disassembly profit and minimize energy consumption, disassembly difficulty, and the number of workstations.

10.1016/j.ifacol.2021.04.076

2) In order to solve this problem, a Multi-Objective Artificial Bee Colony Algorithm (MOABC) with stochastic simulation is designed.

3) By disassembling real products, we compare MOABC with NSGA-II and MDGWO and the results show its effectiveness and feasibility over them in solving the investigated problems.

The rest of this paper is organized as follows. Section II describes the investigated problem. Section III presents a MOABC. Section IV gives the experimental results. Finally, a conclusion is made in Section V.

II. PROBLEM STATEMENT

A. Notation

In order to construct a mathematical model, we introduce the following notations.

1) i: index of a subassembly, $i = 1, 2, ..., N$, where N is the number of subassemblies of an EOL product.

2) j, k: index of a task j, $k = 0, 1, 2, ..., J$, where J represents the number of tasks and 0 is a dummy task.

3) l, m: index of workstations, l, m=1, 2, …, M, where M is the number of workstations.

4) c_j^h: disassembly cost per unit of time of the j-th disassembly task conducted by human h.

5) c_j^r: disassembly cost per unit of time of the j-th disassembly task conducted by a robot r.

6) c_{jk}: setup cost per unit of time of the task k if it follows task j.

7) c_l: task cost of the l-th workstation.

8) D: disassembly-incidence matrix of a given AND/OR graph.

9) d_{ij}: an element in the i-th row and the j-th column of D.

10) e_j^h: disassembly energy consumption per unit of time of the j-th disassembly task conducted by human h.

11) e_j^r: disassembly energy consumption per unit of time of the j-th disassembly task conducted by a robot r.

12) e_{jk}: setup energy consumption per unit of time of the task k if it follows task j.

13) e_l: energy consumption of the l-th workstation.

14) F_c: maximum failure cost during disassembly process.

15) K_j: a variable called d-value to measure the difficulty of j-th disassembly task.

16) q_{jk}: failure probability of task k if it closely follows task j.

17) R_i: degradation rate of subassembly i.

18) r_i: recycling value of subassembly i.

19) S: precedence matrix of an AND/OR graph, where s_{jk} is an element in the j-th row and k-th column of S.

20) T_l: fixed time cycle of the workstations.

21) t_j^h: disassembly time of the j-th disassembly task conducted by human h.

22) t_j^r: disassembly time of the j-th disassembly task conducted by a robot r.

23) t_{jk}: setup time of task k if it follows task j.

24) θ: the probability that the failure cost of a disassembly process satisfies the required minimum probability that the failure cost of a disassembly process is less than its maximum value.

25) α: the weight of disassembly difficulty for the operator.

26) β: the weight of disassembly difficulty for a robot.

Decision variables:

1) x_j: if disassembly task j is performed, x_j=1; otherwise x_j=0.

2) y_{jk}: if disassembly task k is performed after task j, y_{jk}=1; otherwise $y_{jk} = 0$.

3) z_{jl}: if disassembly task j is assigned to the l-th workstation, z_{jl}=1; otherwise z_{jl}=0.

4) u_l: if the l-th workstation is used, u_l=1; otherwise u_l=0.

5) O_j: if the disassembly task j is performed by an operator (K_j>10), O_j=1; if j is performed by the robot (K_j<=10), O_j=0.

B. Problem Description

In order to describe the precedence relations among components, an AND/OR graph in Guo *et al*. (2018) is used in this work. The AND/OR graph of a ballpoint pen is shown in Fig. 1. In the graph, the relationship AND denotes that a parent subassembly can be disassembled into its corresponding child subassemblies by using a disassembly task. For instance, via task 1, <1>1, 2, 3, 4, 5, 6 can be disassembled into <2>1, 2, 3, 4, 5 and task <15>6. An OR relationship means that a parent subassembly can be disassembled by different disassembly tasks which can obtain different child subassemblies. For instance, via task 2, <1>1, 2, 3, 4, 5, 6 can also be disassembled into <3>1, 2, 3, 4, 6 and <14>5. Tasks 1 and 2 that are conflict with each other form an OR relationship. We adopt a feasible disassembly process that satisfies these precedence relationships.

A feasible disassembly task sequence should satisfy the following constraints. According to the AND/OR graph, we define two matrices S and D.

1) A precedence matrix S=[s_{jk}] is constructed as:

$$s_{jk}=\begin{cases}1, \text{ if task } j \text{ can be performed before task } k;\\ -1, \text{ if tasks } j \text{ and } k \text{ conflict with each other;}\\ 0, \text{ otherwise.}\end{cases}$$

2) A disassembly-incidence matrix D=[d_{ij}] is constructed to define the relationship between subassemblies and tasks.

$$d_{ij}=\begin{cases}1, \text{ if subassembly } i \text{ can be obtained by task } j.\\ -1, \text{ if subassembly } i \text{ can be disassembled via task } j.\\ 0, \text{ otherwise.}\end{cases}$$

Disassembly tasks can be divided into human and robot types as shown in Fig. 2. K_j is a variable called d-value to measure the difficulty of the j-th disassembly task. If K_j>10, then the task is assigned to human; and if $K_j \le 10$, then the task is assigned to a robot.

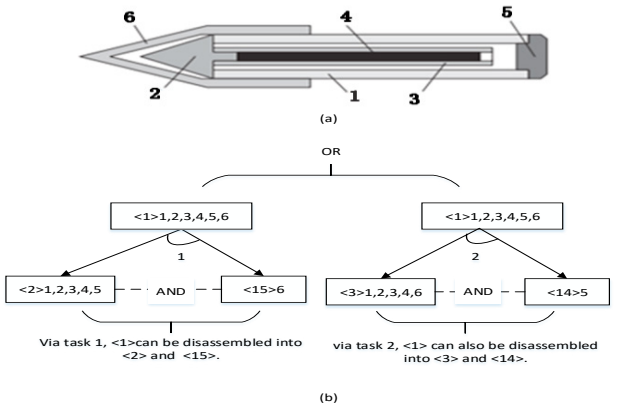

Fig. 1. (a) An example about a ballpoint; (b) A part of AND/OR graph about a ballpoint.

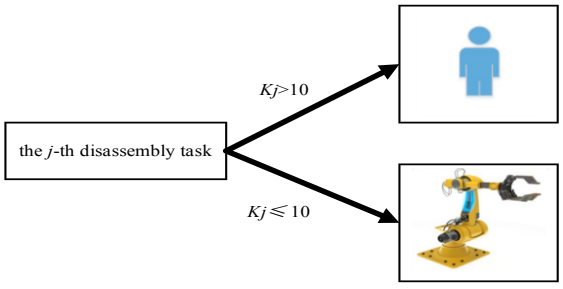

Fig. 2. Classification of disassembly tasks.

HC-DBLP satisfies the following assumptions:

1) The disassembly difficulty of each task is known.

2) Disassembly time and setup time are random variables generated by a normal distribution.

3) There is a priority constraint relationship among disassembly tasks, and S and D are known.

4) The total disassembly time of each workstation should not exceed a fixed cycle time of the workstation.

5) At least one disassembly task is assigned to each workstation used.

6) The workstation starts at the first task and is closed when the last task is completed.

7) Not all components in an EOL product can be disassembled.

C. Mathematical model

$$\max f_1=\mathrm{E}\left(\sum_{j=1}^{J}\sum_{i=1}^{N} d_{ij}r_i(1-R_i)x_j-\sum_{j=1}^{J} t_j^h c_j^h x_j O_j-\sum_{j=1}^{J} t_j^r c_j^r x_j(1-O_j)-\sum_{j=0}^{J}\sum_{k=1}^{J} t_{jk}c_{jk}y_{jk}-\sum_{l=1}^{M} c_l u_l\right) \quad (1)$$

$$\min f_2=E\left(\sum_{j=1}^{J} t_j^h e_j^h x_j O_j+\sum_{j=1}^{J} t_j^r e_j^r x_j(1-O_j)+\sum_{j=0}^{J}\sum_{k=1}^{J} t_{jk}e_{jk}y_{jk}+\sum_{l=1}^{M} e_l u_l\right) \quad (2)$$

$$\min f_3=E\left(\sum_{j=1}^{J} \alpha K_j O_j+\sum_{j=1}^{J} \beta K_j(1-O_j)\right) \quad (3)$$

$$\min f_4=E\left(\sum_{l=1}^{M} u_l\right) \quad (4)$$

$$\sum_{j=1}^{J} x_j \geq 1 \quad (5)$$

$$x_k=\sum_{j=1}^{J} y_{jk},\ k=1, 2, \ldots, J \quad (6)$$

$$x_j=\sum_{l=1}^{M} z_{jl} \leq 1, j=1, 2, \ldots, J \quad (7)$$

$$\sum_{j=1}^{J} x_j z_{jl} \geq 1,\ l=1, 2, \ldots, M \quad (8)$$

$$s_{jk}-y_{jk} \geq 0,\ k=1, 2, \ldots, J \quad (9)$$

$$z_{kl} \geq s_{jk}-\sum_{m=1}^{M} z_{jm}, j,\ k=1, 2 \ldots J,\ l=1, 2, \ldots M \quad (10)$$

$$\sum_{j=0}^{J}\sum_{k=1}^{J}\left(x_k z_{kl} t_k^h\right)O_k+(x_k z_{kl} t_k^r)(1-O_k)+y_{jk}z_{jl}t_{jk} \leq T_l,\ l=1, 2, \ldots, M \quad (11)$$

$$P_r\left\{\sum_{j=0}^{J}\sum_{k=1}^{J}\left(t_k^h c_k^h x_k O_k+t_k^r c_k^r x_k(1-O_k)+y_{jk}t_{jk}c_{jk}\right)q_{jk} \leq F_c\right\} \geq \theta \quad (12)$$

$$x_j, y_{jk}, z_{jl}, u_l, O_j \in \{0,1\}, j, k=1, 2 \ldots J, l=1, 2, \ldots, M \quad (13)$$

Objective function (1) represents the maximum expected the profit to disassemble a product. (2) aims to minimize expected the energy consumption. (3) is to minimize expected dismantling difficulty. (4) is to minimize expected number of workstations. (5) denotes that at least one task excluding task 0 must be performed. Each task can be performed only once according to (6). (7) states that a disassembly task can be assigned to one workstation at most once on a disassembly line. (8) ensures that at least one task is performed in an switched-on workstation. (9) requires that a disassembly sequence must satisfy precedence constraints. (10) means that a workstation can get a task only if the task predecessor has been assigned. (11) states the cycle time restraint to each workstation. (12) requires that the probability that the failure cost of a disassembly process being less than its maximum failure cost is greater than a preset value $\theta > 0$. (13) means the range of decision variables.

III. PROPOSED ALGORITHM

ABC proposed by Kalayci and Gupta. (2013) in 2005 is a kind optimization method that imitates bees' behaviors. It divides the artificial bee colony into three types of bees: employed, onlooker and scout ones. Its main feature is that it needs no unique information but the pros and cons of the problem. From the local optimization behaviour of every artificial bee individual, the optimal global value has finally emerged in the group, which has a faster convergence speed.

A. Solution representation and population initialization

We use $V=(V', V'')$ to represent a solution of HC-DBLP, where $V'=\{o_1, o_2, \cdots, o_J\}$ is an integer string that expresses disassembly task sequence, and o_j is the task sequence index in the j-th position. $V''=\{x_1, x_2, \cdots, x_J\}$ is a binary string indicating whether the task is performed. If the task at the j-th position in V' is performed, $x_j = 1$, otherwise, $x_j = 0$. In this work, the MOABC population is randomly generated and then evaluated by random simulation. We develop an external archive to storage all nondominated solutions during a search process and update it with newly-generated individuals by using Pareto rules in Guo *et al*. (2020a).

B. Stochastic simulation to evaluate a solution

Monte Carlo simulation in Fu *et al*. (2018) is a practical approach to evaluate and calculate stochastic and probabilistic functions. In this work, disassembly and setup time are random variables, and we use stochastic simulation to evaluate and estimate the objective function values of a solution. The procedure of the stochastic simulation algorithm is given in Algorithm 1.

C. Employed bee phase

The employed bees search the neighbourhood of the existing solution to generate new solution. In order to generate a new solution, a precedence preserving crossover (PPX) operator in Kheder *et al*. (2015) is used in a crossover operation, and a position-based mutation (PBM) operator in Algorithm 2 is used in a mutation operation.

Onlooker bee phase

According to Mirjalili *et al*. (2014), the crowding distance operations of onlooker bee is given in Algorithm 3 to make the obtained solutions more uniform in the target space. The greater the crowding distance, the greater the range and variety of solutions. The smaller the crowding distance, the smaller the range of solutions and the less diversity. At the same time, when it is less than a certain range, we use a simulated annealing algorithm to jump out of the local optimal solution space according to Feng *et al*. (2008).

Scout bee phase

If the employed bee fails to improve the quality of the solution after a given number of attempts, the employed bee becomes scout bee, and its solution is abandoned. The transformed scout bee starts a random search for new solutions. Scout bee can jump out of local optimal solutions.

Algorithm 1: The Stochastic simulation algorithm

Input: an initial individual *t*.
Output: a new individual *t*.
Begin
Initialize n, θ
for (i = 0 to n)
  According to the random distribution of t_j^h, t_j^r, and t_{jk}, a sample is generated.
  Calculate the current sample, i.e., f_1-f_4, and the cost of failure.
end for
  n'= $\lfloor\theta \cdot n\rfloor$ and find the n'-th largest sample
 if equation (12) is satisfied **then**
    Return the average of n samples
    else
    Return infeasibility.
 end if
End

Algorithm 2: PBM operator

Input: $V_g=(V'_g,V''_g)$.
Output: $V_g=(V'_g,V''_g)$.
Begin
 if (a randomly-generated number P on an interval [0, 1) is less than P_m).
  Generate a random number t on an interval [0, 2].
  if (t = 0) **then**
    Generate two random numbers v *and* w on an interval $[1, J]$.
    Swap $V'_g(v)$ and $V'_g(w)$.
  else if (t = 1) **then**
    Generate a random number x on an interval $[1, J]$ and $x = x + J$.
      if ($V''_g(x)$ = 1) **then** $V''_g(x)$=0
       else $V''_g(x)$=1
      end if
  end if
  if (t=2) **then**
    Do the task when t = 0 and do the task when t =1.
  end if
 end if
End.

Algorithm 3: The crowding distance operation of onlooker bee

Input: population P
Output:
Begin
 Let the size of the solution set P be L, and P_i be the i-th individual in P.
 population P.
 for (i =0 to L)
  if (the rank of P_i =1)
   Put P_i in solution set P'.
  end if
  Let the size of the solution set P' be N, and P'_i be the i-th individual
  in solution set P'. d_i is the crowded distance of the i-th individual
  in solution set P'.
  if (N>1)
   initialize two variables m and n.
   Rank all individuals from biggest to smallest in P' according to f_0.
   The crowding distance of all individuals are set to 0.
    for (i = 0 to 3)
   m and n are the objective function f_i value in P'_0 and P'_{N-1}, respectively.
  end for
  set variable p, s, d.
    for (j = 1 to N-1)
   p and s are the values of the objective function f_i in P'_{j-1} and P'_{j+1},
    respectively. $d = \sqrt{(p-s)/(m-n)}$ and $d_j = d_j + d$.
  end for
    m is set to the maximum value of the crowded distance of all
    individuals in P'. d_0=m, d_{N-1}=m.
    for (i = 0 to N)
     d_i += 0.000001
    end for
  d is the sum of all crowded distance of individuals in P'.
  $d_i = d_i / d$.
  d_i is set to the sum of crowding distance of two neighboring individuals.
 end if
End.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. Case study

In order to test the effectiveness and feasibility of the proposed algorithm, we choose a hammer drill to study it. The AND/OR graph of a hammer drill can be constructed according to the priority relation in Pistolesi *et al*. (2018). We have simplified the parts appropriately. The fixed cost, energy and cycle time of each workstation are set to 70, 100, and 70, respectively. In addition, the F_c and θ are set to 60 and 0.95, respectively. For this problem, each algorithm runs 20 times. In this work, NSGA-II and MDGWO are selected as algorithms for comparison. NSGA-II is a simple, efficient and widely-used multi-objective evolutionary algorithm. It proposes a fast-non-dominated sort method that improves the algorithm convergence speed. MDGWO has fast convergence and can effectively obtain many Pareto solutions. All algorithms are implemented in IntelliJ IDEA 2018.2.5 × 64 and run on an Intel (R) Core (TM) i7-6700HQ CPU (2.60GHz / 8.00GB RAM) PC with Windows 10 operating system. We list 8 non-dominated solutions from MOABC in Table 1, where f_1-f_4 and f_c represent total profit, energy consumption, disassembly difficulties, the number of workstations, and failure cost, respectively. Please note that we use stochastic simulation to obtain its approximate objective function.

Table 1. Disassembly Sequence of Hammer Drill Obtained By MOABC

	Disassembly decisions	f_1	f_2	f_3	f_4	f_c
1	1 3 6 8 5 11 7 12 13 18 16 9 19 20	933.3	1625.6	120.2	3	39.6
2	1 3 6 8 7 11 12 13 5 9 16	798.6	1247.0	80.0	3	24.8
3	1 3 6 7 8 5 11 12 9 14 18 17 20	1013.6	1636.6	130.8	3	40.5
4	1 3 6 8 11 7 5 12 18 13 16 19 9 20	998.3	1649.6	120.2	3	52.0
5	2 4 6 8 11 12 13 16 18 7 9 19 20	921.9	1603.3	131.6	3	43.0
6	2 4 6 8 7 11 12 14 18 9 19 21 17 22 24	1190.8	1976.0	181.6	3	50.6
7	2 4 6 8 7 11 12 18 9 19 13 16 21 22 20 24	1219.7	2010.3	171.0	4	58.9
8	2 4 6 7 8 11 5 9 12 18 14 19 21 17 20 22 24	1305.4	2207.0	195.2	4	56.7

Through the number of Pareto optimal solutions, CPU average running time and Inverted Generational Distance (IGD) in Geng *et al*. (2016), the performance of three algorithms in solving the problem is compared. The indexes and its graphic content are described as follows.

1) IGD: The value of IGD decides convergence and distribution of the algorithm. Here, each algorithm runs 20 times to get sets of IGD. Through the mean and variance of IGD, $|t|$ can be obtained among them. To analyze the effectiveness of experimental results, a *t*-test with 38 degrees of freedom and 0.05 significance level is used. In Table 2, "+" and "-" indicate that the mean value of IGD obtained by MOABC is significantly better than and less than its peer, respectively. "~" indicates that $|t| < t_{0.05}(38)$, MOABC is equivalent to its peer.

2) CPU average running time: It is used to compare the CPU average running time of each algorithm. The CPU average running time is obtained by calculating the average time of each algorithm executing 20 times as shown in Table 3.

3) The number of Pareto optimal solutions: It is used to compare the number of solutions obtained by each algorithm. The horizontal axis and the vertical axis in Figs. 3 represent the 1st to 10th execution of an algorithm and the size of the Pareto solution set obtained by each algorithm.

Table 2. Comparison of Three Algorithms via IGD-metric

algorithm	IGD-metric		
	mean	variance	*t*-test
MOABC	0.0509	1.8923×10^{-4}	null
MDGWO	0.0663	2.3103×10^{-4}	+
NSGA-II	0.1262	1.3651×10^{-3}	+

Table 3. CPU Average Running Time Index

	CPU average running time index	
	Algorithm name	***Case study***
1	MOABC	4715.80*ms*
2	MDGWO	4826.65*ms*
3	NSGA-II	4704.65*ms*

Fig.3. A quantitative index of Case study.

B. Analysis of results.

From the above experimental results, we can conclude that:

1) In terms of IGD, in case study, MOABC can obtain the minimum mean value, which shows that MOABC has better convergence and diversity than its two peers. The results of a *t*-test show that MOABC significantly outperforms NSGA-II and MDGWO.

2) In terms of CPU average running time, in Case study, CPU average running time of MOABC is shorter than MDGWO and longer than NSGA-II.

3) In terms of the number of Pareto optimal solutions, in case study, MOABC has obtained more solutions than NSGA-II and MDGWO, thus providing more solutions for decision-makers and ensuring better diversity.

V. CONCLUSION

In this paper, a method of stochastic simulation is combined with the basic ABC to ensure profit maximization, and minimization of energy consumption, dismantling difficulty and number of workstations. By considering the risk of task failure, the decision-maker can well avoid the loss caused by the failure of disassembly. Considering human-robot cooperation, the efficiency of disassembly can be improved. The results show that MOABC is superior over NSGA-II and MDGWO. In the future, we will develop more powerful intelligent optimization algorithms to solve more practical problems, e.g. Bi *et al*. (2020), Guo *et al*. (2020b).

ACKNOWLEDGMENT

This work is supported in part by Liaoning Province Education, Department Scientific Research Foundation of China under Grand No.L2019027; Liaoning Revitalization Talents Program under Grant No. XLYC1907166; The Natural Science Foundation of Shandong Province under Grant ZR2019BF004; National Natural Science Foundation of China (61573089); The 13th five-year plan on education and science （JG18DA013）.

REFERENCES

Bi, J. Yuan, H.T. Duanmu, S. F. Zhou M.C. Abusorrah. and A. (2020) Energy-optimized Partial Computation Offloading in Mobile Edge Computing with Genetic Simulated-annealing-based Particle Swarm Optimization, *IEEE Trans. Auto. Sci. Eng.,* DOI 10.1109/JIOT.2020.3024223, 2020.

Feng, J., Shi, J.S., Cheng, Wu., (2008) "A Simulated Annealing Algorithm for Single Machine Scheduling Problems with Family Setups," *COMPUT OPER RES*, dio: 10.1016/j.cor.2008.08.001.

Fu, Y.P., Wang, H.F., Huang, M., (2018) "Integrated scheduling for a distributed manufacturing system: a stochastic multi-objective model," *Enterp. Inform. Syst.*, doi: 10.1080/17517575.2018.1545160.

Fu, Y.P., Zhou, M.C., Guo, X.W., and Qi, L., (2019) "Artificial-molecule-based chemical reaction optimization for flow shop scheduling problem with deteriorating and learning effects," *IEEE Access*., vol. 7, pp. 53429-53440.

Geng, H.T., Li, H.J., Zhao, Y.G., Chen, Z.P., (2016) "Improved NSGA-II algorithm based on adaptive hybrid non-dominated individual sorting strategy," *J. Comput. Appl*., vol. 36, no. 5, pp.1319-1324+1340.

Guo, X.W., Liu, S. X., Zhou, M.C., and Tian, G. D., (2018) "Dual-objective program and scatter search for the optimization of disassembly sequences subject to multiresource constraints," *IEEE Trans. Auto. Sci. Eng.*, vol. 15, no. 3, pp. 1091-1103.

Guo, X.W., Zhou, M.C., Liu, S.X., and Qi, L., (2019) "Lexicographic multiobjective scatter search for the optimization of sequence-dependent selective disassembly subject to multiresource constraints," *IEEE Trans. Cybern*., doi: 10.1109/TCYB.2019.2901834.

Guo, X.W., Zhou, M.C., Alsokhiry, F. and Sedraoui, K., (2020a) "Disassembly sequence planning: a survey," *IEEE/CAA J. Autom. Sinica.* in press.

Guo, X.W., Zhou, M.C., Liu, S.X., and Qi, L., (2020b) "Multi-resource constrained selective disassembly with maximal profit and minimal energy consumption," *IEEE Trans. Auto. Sci. Eng*., in press.

Huang, L., Zhou, M., Hao, K. R., and Hou, E., (2019) "A survey of multi-robot regular and adversarial patrolling," IEEE/CAA J. Autom. Sinica, vol. 6, no. 4, pp. 865-874.

Ilgin, M.A., Akçay, H., and Araz, C., (2017) "Disassembly line balancing using linear physical programming," *Int. J. Prod. Res*., vol. 55, no. 20, pp. 6108-6119.

Kalayci, C.B. and Gupta, S.M., (2013) "Artificial bee colony algorithm for solving sequence-dependent disassembly line balancing problem," *Expert Syst. Appl*., vol. 40, no.18, pp.7231-7241.

Kalayci, C.B., Hancilar, A., Gungor, A., and Gupta, S.M., (2015) "Multi-objective fuzzy disassembly line balancing using a hybrid discrete artificial bee colony algorithm," *J. Manuf. Syst*., vol. 37, pp. 672-682.

Kheder, M., Trigui, M., and Aifaoui, N., (2015) "Disassembly sequence planning based on a genetic algorithm," *P. I. Mech. Eng. C-J Mec*., vol. 229, no. 12, pp. 2281-2290.

Li, K., Liu, Q., Xu, W.J., Liu, J.Y., Zhou, Z.D., and Feng. H., (2019) "Sequence planning considering human Fatigue for Human-Robot Collaboration in disassembly," *Procedia CIRP*., doi: 10.1016/j.procir.2019.04.127.

Liu, J. Y., Zhou, Z. D., Pham, D. T., Xu, W.J., Yan, J.W., Liu, A.M., Ji, C.Q., and Liu, Q., (2018) "An improved multi-objective discrete bees algorithm for robotic disassembly line balancing problem in remanufacturing," *Int. J. Adv. Manuf. Tech*., doi: 10.1007/s00170-018-2183-7.

McGovern, S.M., and Gupta, S.M., (2007) "A balancing method and genetic algorithm for disassembly line balancing," *Eur. J. Oper. Res*., vol. 179, no. 3, pp. 692-708.

Mirjalili, S., Mirjalili, S.M., Lewis, A., (2014) "Grey wolf optimizer," *Adv. Eng. Soft*., vol. 69, pp. 46-61.

Özceylan, E., Kalayci, C.B., Güngör, A., and Surendra, M.G., (2018) "Disassembly line balancing problem: a review of the state of the art and future directions," *Int. J. Prod. Res*., doi: org/10.1080/00207543.2018. 1428775.

Pistolesi, F., Lazzerini, B., Dalle Mura, M. and Dini, G., (2018) "EMOGA: A hybrid genetic algorithm with extremal optimization core for multobjective disassembly line balancing," *IEEE Trans. Ind. Inform*., vol. 14, no. 3, pp. 1089-1098.

Tian, G.D., Zhou, M.C., Chu, J.W., and Liu, Y.M., (2012.) "Probability evaluation models of product disassembly cost subject to random removal time and different removal labor cost," *IEEE Trans. Autom. Sci. Eng*., vol. 9, no. 2, pp. 288-295, Apr.

Wegener, K., Chen, W.H., Dietrich, F., Dröder, K., and Kara, S., (2015) "Robot assisted disassembly for the recycling of electric vehicle batteries," *Procedia CIRP*, pp. 716-721, doi: 10.1016/j.procir.2015.02.051.

Xiong, W. Q., Pan, C. R., Qiao, Y., Wu, N. Q., Chen, M. X., and P. H. Hsieh, (2020) "Reducing wafer delay time by robot idle time regulation for single-arm cluster tools," IEEE Trans. Auto. Sci. Eng., DOI: 10.1109/TASE.2020.3014078.

Zhang, Z.Q., Wang, K. P., Zhu, L. X., and Wang, Y., (2017) "A Pareto improved artificial fish swarm algorithm for solving a multi-objective fuzzy disassembly line balancing problem," *Expert Syst. Appl*., doi: 10.1016/j.eswa.2017.05.053.

Zheng, F.F., He, J.K., Chu, F., and Liu, M., (2018) "A new distribution-free model for disassembly line balancing problem with stochastic task processing times," *Int. J. Prod. Res*., doi: 10.1080/00207543.2018.1430909.